Glowy

Dossier P3 : Recherche et réservation

Projet d'équipe · octobre 2026

Ce dossier explique le projet en détail, puis **ta gestion** : ce qu'elle fait, ses règles métier, son IA, les fichiers à connaître et comment la présenter à la validation.

Le projet Glowy en détail

1. Qu'est-ce que Glowy ?

Glowy est une **plateforme de réservation de salons de beauté vérifiés en Tunisie** (coiffure, barbier, ongles, spa, visage…). Une cliente cherche un salon, voit les prestations, les prix en dinars, les avis et l'affluence, puis réserve un créneau précis en moins de 2 minutes. Le salon gère tout son travail depuis un tableau de bord, et un Super Admin contrôle la plateforme.

- **4 rôles** : cliente, employé, gérant de salon, Super Admin.
- **3 langues** : français, arabe (écriture de droite à gauche) et anglais.
- **Application installable** sur téléphone (PWA), utilisable hors connexion.
- **5 intelligences artificielles**, une par membre de l'équipe.
- Projet d'équipe de 5 étudiants : chaque personne a **1 gestion classique + 1 IA**.

2. Les technologies

Domaine	Technologie	Rôle dans le projet
Langage	**TypeScript**	partout : pages, API, règles métier, tests
Framework	**Next.js 16** (App Router) + **React 19**	site et API dans un seul projet
Interface	**Tailwind CSS 4**, composants shadcn / Base UI	mise en page responsive, mobile first
Animations	**GSAP**	apparitions et transitions
Base de données	**MongoDB** + **Mongoose 9**	33 collections (comptes, salons, réservations, avis, points…)
Authentification	**Auth.js v5**	connexion e-mail / mot de passe, Google, sessions sécurisées
Validation	**Zod**	contrôle de toutes les données reçues par l'API
Carte	**Leaflet** + OpenStreetMap	carte des salons
Graphiques	**Recharts**	statistiques du dashboard
E-mails	**nodemailer** (SMTP)	mot de passe oublié, rappels

Domaine	Technologie	Rôle dans le projet
Notifications	**Web Push**	rappels sur le téléphone
Tests	**node:test** + tsx	75 tests automatisés sur une base de test

3. L'architecture

Le code suit une **architecture propre (Clean Architecture)** : chaque domaine est un **module** indépendant dans `src/modules/<module>/`, découpé en 3 couches.

Couche	Contenu	Exemple
`domain`	règles métier pures, sans base de données, faciles à tester	calcul de l'acompte, verrou des créneaux
`application`	cas d'usage et validation des données (Zod)	« créer une réservation »
`infrastructure`	lecture et écriture dans MongoDB	requêtes Mongoose

Organisation des dossiers :

```
src/
  app/              pages et routes API (Next.js)
    (site)/         site public + espace client (/compte)
    (auth)/         connexion, inscription, mot de passe oublié
    dashboard/      espace du salon (gérant, employés)
    admin/          espace Super Admin
    api/            routes REST
  modules/          un dossier par domaine (identity, booking, reviews, loyalty…)
  components/       composants d'interface
  core/domain/      règles communes (erreurs, heure de Tunis, événements)
  infrastructure/   connexion MongoDB, sessions, outils HTTP
  i18n/             traductions français / arabe / anglais
public/sw.js        service worker (hors connexion, notifications)
tests/              tests automatisés
```

Les modules communiquent par événements. Exemple : quand le salon marque un rendez-vous « terminé », le module réservation publie `booking.completed`. Le module fidélité y réagit en créditant les points, et le module notifications prévient la cliente. Aucun module ne modifie le code d'un autre.

Tâches planifiées (toutes les 10 minutes) : rappels de rendez-vous, préavis et expiration des points, analyse des nouveaux avis.

4. La base de données

MongoDB stocke les données sous forme de documents. Les données qui vivent toujours avec leur parent sont **imbriquées** : les horaires et les prestations dans le salon, les lignes et le paiement dans la réservation, le compte fidélité dans l'utilisateur.

Collection	Contenu
`User`	comptes (rôle, mot de passe haché, points, favoris)
`Salon`	salons (adresse, horaires, photos, prestations, plan)
`Employee`, `TimeOff`	équipe et congés

Collection	Contenu
`Booking`	réservations (prestations, statut, acompte)
`SlotLock`	verrou anti double réservation (tranches de 5 min)
`Review`, `ReviewReport`	avis et signalements
`PointTransaction`, `Reward`, `Redemption`	fidélité
`Notification`, `PushSubscription`	notifications
`PlanPayment`	abonnements des salons
`AuditLog`	journal des actions sensibles

5. La sécurité

- Mots de passe hachés (bcrypt) ; compte bloqué 15 minutes après 5 échecs.
- Rôle vérifié sur **chaque page et chaque API**.
- Protection CSRF : une écriture venant d'un autre site est refusée.
- Toutes les données reçues sont validées (Zod) avant d'arriver en base.
- Les actions sensibles du Super Admin sont journalisées.

6. L'application mobile (PWA)

- **Manifest** : nom, icônes, mode plein écran, raccourcis. L'application s'installe sur l'écran d'accueil.
- **Mode hors connexion** : le service worker enregistre les pages consultées et précharge les pages publiques liées. Sans réseau, elles s'ouvrent depuis le téléphone ; sinon la page `/offline` liste les pages disponibles. Les pages personnelles sont effacées à la déconnexion.
- **Mobile first et responsive** : testé de 320 à 1440 px de large, en français, en arabe et en anglais, sans aucun débordement.

7. Lancer le projet

```
npm install
npm run db:seed        données de démonstration
npm run dev            développement : http://localhost:3000
npm run build && npm start      production (nécessaire pour le hors connexion)
npm test               75 tests automatisés
```

Sur la page de connexion, les boutons **« Accès démo par rôle »** connectent en un clic : Client, Gérant, Employé, Super Admin.

8. Les 5 gestions

#	Gestion	Responsable	IA associée
1	Comptes, sécurité et Super Admin	P1	Recommandation de salons
2	Dashboard du salon	P2	Prédiction d'affluence
3	Recherche et réservation	P3	Assistante de réservation
4	Avis, modération et support	P4	Analyse des avis

#	Gestion	Responsable	IA associée
5	Espace client, fidélité, abonnements et PWA	P5	Analyse beauté par photo

Un parcours complet relie les 5 gestions :

1. La cliente se connecte et voit des salons recommandés (gestion 1).
2. Elle réserve un créneau (gestion 3).
3. Le salon voit le rendez-vous dans son agenda et le marque terminé (gestion 2).
4. La cliente gagne des points (gestion 5) et laisse un avis vérifié, analysé par l'IA (gestion 4).
5. Le Super Admin suit l'activité et modère (gestion 1).

3 Gestion de la recherche et des réservations (P3)

But : permettre à une cliente de trouver un salon et de réserver un créneau **sans jamais de double réservation**.

Fonctionnalités

- **Recherche** par ville, prestation, note, prix et disponibilité du jour, avec une **carte** OpenStreetMap.
- **Fiche salon** : prestations et prix, équipe, horaires, photos, avis, affluence en direct.
- **Tunnel de réservation** : prestations, puis professionnel (ou « peu importe »), puis créneau, puis paiement. Le paiement se fait sur place, ou par acompte en ligne en mode test.
- **Devis** avant confirmation : total, acompte, code promo, offre de dernière minute.
- **Annulation et report** selon la politique du salon. Au-delà du délai fixé par le salon, l'annulation compte comme tardive.
- **Suivi par le salon** : en cours, terminé, absence.
- **Rappels** par e-mail, SMS et notification la veille puis 2 heures avant.
- **Liste d'attente** d'un jour complet : la cliente est prévenue si une annulation libère un créneau qui lui convient.
- **File d'attente virtuelle** : la cliente prend sa place depuis la fiche du salon et suit sa position.

Règles clés

- **Anti double réservation** : chaque réservation verrouille ses tranches de 5 minutes dans la collection `SlotLock`, protégée par un index **unique** (employé + tranche). Si deux réservations arrivent en même temps, une seule passe. Un test le vérifie : 10 réservations simultanées du même créneau, une seule acceptée.
- **Créneaux et délais** :
  - créneaux proposés toutes les 30 minutes ;
  - impossible de réserver moins de 15 minutes à l'avance.
- **Acompte en ligne** :
  - 20 % du total par défaut, réglable par le salon ;
  - borné entre 10 et 30 DT, et jamais plus que le total.

IA — Assistante de réservation

- **Entrée** : une phrase libre en français, en arabe (darija courante comprise) ou en anglais. Par exemple : « je veux une coupe samedi après-midi à La Marsa ».
- **Méthode** : l'assistante repère la prestation, la date, le moment de la journée, l'heure et la ville. Elle répond aussi aux questions fréquentes. Elle cherche ensuite de **vrais créneaux libres** dans les salons les mieux adaptés, dans l'ordre de la recommandation de la gestion 1.
- **Sortie** : des créneaux réels proposés. La réservation n'est faite **qu'après confirmation explicite**, par l'API de réservation habituelle.
- **Repli** : une demande non comprise renvoie vers le formulaire de réservation.
- **Code** : `src/modules/assistant/domain/nlu.ts` · `POST /api/ai/chat`.

Repères techniques

Modules	`booking`, `catalog`, `assistant`
Pages	`/` (accueil), `/salons`, `/salons/[slug]`, `/salons/[slug]/reserver`, fenêtre de l'assistante
API	`/api/salons/*` (recherche, fiche, disponibilités, devis, liste d'attente), `/api/bookings/*`, `/api/ai/chat`
Collections	`Booking` (prestations et paiement imbriqués), `SlotLock`, `WaitlistEntry`
Événements émis	`booking.created`, `booking.cancelled`, `booking.completed`, `booking.noShow`
Tests	`booking`, `waitlist`, `assistant`, `scheduler`

Ma préparation pour la validation

Fichiers à connaître

- `src/modules/booking/` (réservation, verrou, liste d'attente), `src/modules/catalog/` (recherche, fiche salon), `src/modules/assistant/`
- Pages : `src/app/(site)/salons/*`, accueil · API : `src/app/api/salons/*`, `src/app/api/bookings/*`
- Composants : `src/components/booking/*`, `src/components/salon/*`
- Tests : `tests/booking.test.ts` (dont 10 réservations simultanées), `tests/waitlist.test.ts`, `tests/assistant.test.ts`

Démo (3 minutes)

1. **Recherche** `/salons` : filtres, tri, carte, pagination (6 salons par page).
2. **Réservation** (Create) : fiche salon → prestation → professionnel → créneau → confirmation.
3. **Mes rendez-vous** (Read) puis **report** (Update) et **annulation** (Delete logique : l'historique est gardé).
4. Côté salon : le rendez-vous apparaît dans l'agenda, statut « en cours » puis « terminé ».

Montrer l'IA

- Où la montrer : bouton « Réserver avec l'assistante ».

Règles métier à savoir expliquer

- **Anti double réservation** : chaque réservation verrouille ses tranches de 5 min dans `SlotLock`, avec un index **unique** (employé + tranche) ; si deux réservations arrivent en même temps, une seule passe.
- Réservation au moins 15 min à l'avance ; créneaux toutes les 30 min.
- Acompte en ligne : 20 % du total (réglable), entre 10 et 30 DT.
- Annulation tardive selon le délai fixé par le salon ; rappels la veille et 2 h avant.

Questions probables du jury

- *Pourquoi un index unique plutôt qu'une vérification dans le code ?* Deux requêtes simultanées passeraient toutes les deux la vérification ; la base, elle, refuse la seconde.
- *Pourquoi ne pas supprimer une réservation annulée ?* Pour l'historique, les statistiques et la politique d'annulation.